

# Control Before Consequence: Runtime Admissibility for Safe Agentic AI Systems

**Dr. Torsten Bernasco**

Independent Researcher & Chief AI Architect  
Düsseldorf, Germany  
drtorstenbernasco@gmail.com

## Abstract

As AI systems evolve from information-processing tools into increasingly autonomous actors, governance challenges shift from understanding generated content to controlling consequential action. Existing governance mechanisms provide important capabilities for transparency, accountability, auditing, compliance, and safety evaluation. However, these mechanisms frequently operate before action generation or after action execution, leaving a governance gap at the point where AI-generated actions become executable.

This paper introduces the Control Before Consequence framework and proposes Runtime Admissibility as a governance capability for determining whether a proposed action may become externally consequential under current constraints. Runtime admissibility establishes an execution-control boundary between agent reasoning and action execution by evaluating proposed actions against system state and applicable constraints before execution occurs.

We formalize runtime admissibility as a decision function operating over system state, proposed actions, and constraint sets, producing one of three outcomes: ALLOW, BLOCK, or REQUIRE\_APPROVAL. We further present a Runtime Admissibility Gate MVP that demonstrates the concept through reference applications including email actions, record updates, and API invocations.

The paper argues that safe agent ecosystems require governance mechanisms that operate at runtime rather than exclusively during development, evaluation, or post-execution review. We therefore propose runtime admissibility as a foundational roadmap capability for trustworthy agentic AI systems and discuss its implications for future execution-control architectures, policy frameworks, and governance standards.

**Keywords:** Agentic AI, AI Governance, Runtime Admissibility, Control Before Consequence, Safe Agents, Execution Control, AI Safety.

## 1. Introduction

Artificial intelligence systems are rapidly evolving from information-processing tools into increasingly autonomous actors capable of initiating and executing actions in the external world.

Recent advances in large language models, multimodal systems, agent architectures, tool use, workflow orchestration, and autonomous planning have significantly expanded the operational capabilities of AI systems. These systems are no longer limited to generating content. They can initiate workflows, invoke APIs, modify records, communicate with external parties, and interact with critical infrastructure.

As agency increases, the governance challenge changes fundamentally.

Historically, AI governance has focused on model development, transparency, fairness, accountability, safety evaluation, and regulatory compliance. These mechanisms remain essential. However, many existing governance approaches assume a separation between reasoning and action that becomes increasingly difficult to maintain in highly autonomous systems.

The central challenge is therefore no longer limited to what AI systems know or generate. It increasingly concerns what AI systems are allowed to do and under what conditions their actions may become consequential.

This paper argues that safe agentic AI systems require a new governance capability that operates at the boundary between agent reasoning and action execution.

We introduce the Control Before Consequence framework and propose Runtime Admissibility as the decision process that determines whether a proposed action may become externally consequential under current constraints.

To illustrate this concept, we present a Runtime Admissibility Gate MVP that implements a minimal execution-control layer capable of producing admissibility decisions before action execution occurs.

The paper makes four contributions:

1. It identifies a governance gap between agent reasoning and consequential action execution.
2. It introduces Runtime Admissibility as a distinct execution-control capability.
3. It presents a formal model for admissibility evaluation.
4. It demonstrates the concept through a working Runtime Admissibility Gate MVP.

We conclude by discussing the implications of runtime admissibility for future trustworthy agentic AI roadmaps and governance architectures.

## 2. Limitations of Current Governance Paradigms

The rapid advancement of agentic AI systems challenges many assumptions underlying current AI governance approaches.

Existing governance frameworks have made significant progress in addressing concerns related to transparency, accountability, fairness, robustness, security, compliance, and oversight. Examples include the NIST AI Risk Management Framework (NIST, 2023), the European Union AI Act (European Union, 2024), and emerging trustworthy AI initiatives across industry and academia. Regulatory initiatives, standards, and risk management frameworks increasingly provide guidance for the responsible development and deployment of AI systems.

However, many of these approaches were developed in a context where AI systems primarily generated information rather than initiated actions.

As a result, governance mechanisms often focus on three stages:

- model development
- model evaluation
- post-execution accountability

These mechanisms are necessary but increasingly insufficient for highly autonomous systems capable of initiating externally consequential actions.

### 2.1 Governance Does Not Automatically Translate Into Runtime Control

A system may satisfy governance requirements while still executing actions that should not occur under specific runtime conditions.

Transparency may explain why an action occurred.

Auditing may record that an action occurred.

Accountability may determine responsibility after an action occurred.

None of these mechanisms necessarily determine whether the action should have been allowed to execute in the first place.

This distinction becomes increasingly important as agentic systems acquire greater operational autonomy.

### 2.2 Deterministic Assumptions Break Under Agentic Behavior

Many governance approaches implicitly assume predictable workflows and relatively stable operational environments.

Agentic systems introduce new characteristics including:

- autonomous planning
- dynamic tool selection
- adaptive workflows
- interaction with external systems
- evolving execution contexts

Under such conditions, admissibility cannot always be fully determined during system design.

Execution decisions increasingly depend on runtime context.

## 2.3 The Missing Layer

Current governance architectures often lack an explicit execution-control layer positioned between agent reasoning and consequential action.

As agent capabilities continue to expand, this missing layer becomes a significant governance gap.

The challenge is not simply to determine whether an action is technically feasible.

The challenge is to determine whether an action should be permitted to become consequential under current constraints.

This gap motivates the need for Runtime Admissibility and the broader Control Before Consequence framework presented in the following sections.

## 3. Control Before Consequence

### 3.1 From Observation to Intervention

The governance of AI systems has traditionally focused on observation, monitoring, auditing, and post-hoc accountability. These mechanisms remain essential, but they are increasingly insufficient for agentic systems capable of initiating externally consequential actions.

As AI systems evolve from information-processing systems into action-producing systems, governance must extend beyond observing outcomes and toward controlling execution. The critical challenge is no longer limited to understanding what an AI system generated, but determining whether a proposed action should be permitted to affect the external world.

This shift represents a transition from observational governance toward intervention-oriented governance. Rather than evaluating actions after consequences occur, safe agentic systems require mechanisms capable of evaluating admissibility before execution.

The central question therefore becomes:

*Under what conditions should an AI-generated action be allowed to become consequential?*

This question defines a new control problem for agentic AI systems.

### 3.2 Runtime Admissibility

We introduce Runtime Admissibility as the decision process that determines whether a proposed action may become externally consequential under the current constraints.

Runtime admissibility operates at the boundary between agent reasoning and action execution. It evaluates a proposed action against the current system state and the applicable constraints before execution is permitted.

Unlike traditional guardrails, which primarily focus on inputs, outputs, or generated content, runtime admissibility focuses on execution itself. Related approaches have focused on model safety, alignment, transparency, and risk mitigation (Amodei et al., 2016; Rossi, 2019). Runtime admissibility focuses on a different question: whether a proposed action should be allowed to execute under current runtime constraints. The objective is not to determine whether

generated content is acceptable, but whether a proposed action should be allowed to cause a state transition in the external environment.

Runtime admissibility therefore introduces an explicit execution-control boundary between agent cognition and consequential action.

Within the Control Before Consequence framework, no externally consequential action should execute before admissibility has been resolved.

This principle establishes runtime admissibility as a foundational capability for safe and trustworthy agentic AI systems.

### 3.3 Formal Model

Runtime admissibility can be represented as a decision function operating over system state, proposed actions, and applicable constraints.

Let:

$$s \in S$$

denote the current system state,

$$a \in A$$

denote a proposed action,

and

$$C$$

denote the applicable constraint set.

Runtime admissibility is defined as:

$$\text{Adm} : S \times A \times C \rightarrow D$$

where:

$$D = \{\text{ALLOW}, \text{BLOCK}, \text{REQUIRE\_APPROVAL}\}$$

The admissibility function evaluates whether a proposed action may become externally consequential under the current constraints.

Execution is permitted only if admissibility evaluates to ALLOW:

$$\text{execute}(a) \Leftrightarrow \text{Adm}(s,a,C) = \text{ALLOW}$$

Consequently:

$$\text{Adm}(s,a,C) \neq \text{ALLOW}$$

implies:

$$\neg \text{execute}(a)$$

This formulation establishes a formal execution-control boundary between agent reasoning and externally consequential actions.

### 3.4 Decision Space

The admissibility decision space consists of three possible outcomes.

**ALLOW.**

The proposed action satisfies the applicable constraints and may execute immediately.

**BLOCK.**

The proposed action violates one or more constraints and must not execute.

**REQUIRE\_APPROVAL.**

The proposed action cannot be automatically admitted and requires human or supervisory review before execution.

The decision space can therefore be represented as:

$$D = \{\text{ALLOW}, \text{BLOCK}, \text{REQUIRE\_APPROVAL}\}$$

This structure separates admissibility evaluation from execution and creates an explicit control layer between agent cognition and consequential action.

The Control Before Consequence principle can therefore be expressed as:

$$\forall a : \text{execute}(a) \Rightarrow \text{Adm}(s,a,C) = \text{ALLOW}$$

In other words:

*No externally consequential action should execute unless admissibility has been explicitly resolved.*

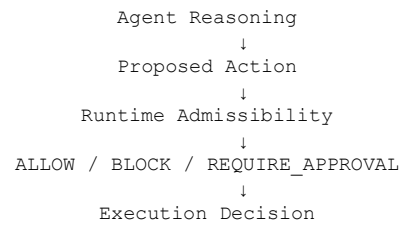


Figure 1. Control Before Consequence Framework

## 4. Runtime Admissibility Gate MVP

An accompanying reference implementation is maintained in the admissibility-gate-mvp repository and serves as a lightweight prototype for illustrating the execution-control boundary.

### 4.1 Architecture

To demonstrate the practical applicability of runtime admissibility, we implemented a lightweight Runtime Admissibility Gate MVP.

The prototype introduces an execution-control boundary between an AI agent and externally consequential actions.

The architecture consists of four primary components:

1. Proposed Action
2. Policy Engine
3. Execution Decision
4. Decision Ledger

The execution flow is illustrated below:

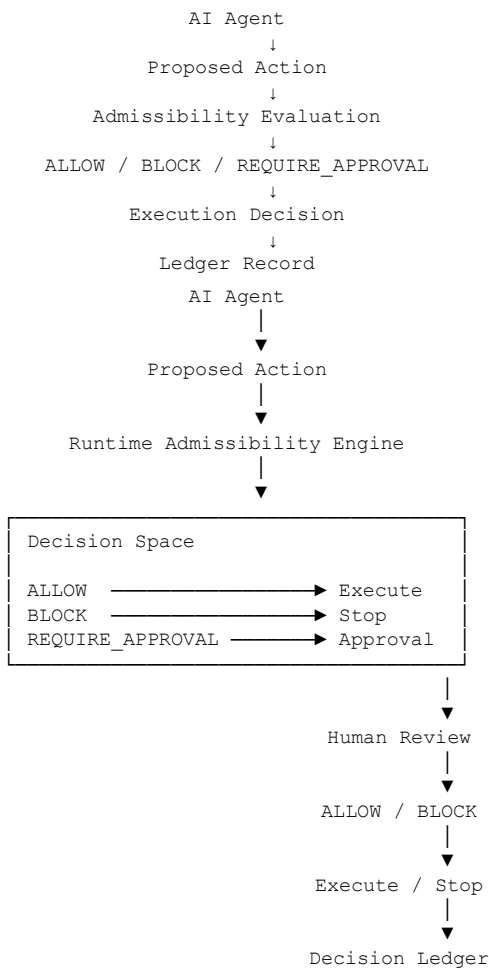


Figure 2. Runtime Admissibility Gate MVP Architecture

The gate evaluates admissibility before execution occurs. Only actions that satisfy admissibility requirements may proceed to execution. Actions that violate constraints are blocked, while actions requiring additional review are suspended pending approval. The objective of the MVP is not to provide a complete governance platform but to demonstrate the execution-control boundary proposed by the Control Before Consequence framework.

## 4.2 Policy Engine

The policy engine represents the decision-making component of the Runtime Admissibility Gate.

It evaluates proposed actions against applicable constraints and produces one of three admissibility decisions:

- ALLOW
- BLOCK
- REQUIRE\_APPROVAL

The policy engine serves as the implementation layer of the runtime admissibility function:

$$\text{Adm}(s,a,C)$$

Within the MVP, policy evaluation is intentionally simple and transparent.

The purpose is not to optimize policy complexity but to demonstrate how admissibility decisions can be enforced before execution.

Different action types may be evaluated using different policies while sharing the same admissibility decision space. This design allows the framework to scale across heterogeneous actions while maintaining a consistent execution-control model.

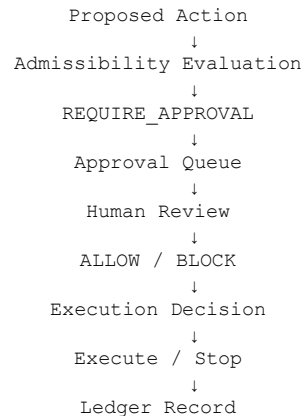
## 4.3 Approval Queue

The Runtime Admissibility Gate may return a decision of REQUIRE\_APPROVAL.

In the current MVP, actions requiring approval are suspended and not executed.

Conceptually, a future approval queue would store pending actions until a human or supervisory process resolves the admissibility decision.

The approval flow can be represented as:



The approval queue does not replace runtime admissibility. Instead, it extends the REQUIRE\_APPROVAL branch of the admissibility decision space.

Actions that cannot be automatically admitted are temporarily suspended until additional review is completed. This mechanism allows governance controls to intervene before externally consequential actions occur.

## 4.4 Reference Applications

The MVP currently implements three reference applications that demonstrate runtime admissibility across different classes of consequential actions.

### Email Actions.

The first reference application evaluates email actions.

Examples include:

- internal communication
- external communication
- sensitive information transfer

Depending on the context, the admissibility gate may allow, block, or require approval before execution.

### Record Updates.

The second reference application evaluates record modification actions.

Examples include:

- customer record updates
- operational data modifications
- workflow state changes

Higher-risk modifications may require approval before execution.

### API Calls.

The third reference application evaluates API invocation requests.

Examples include:

- service integrations
- workflow orchestration
- external system interactions

Risk-sensitive API calls may require approval before execution.

Together, these examples demonstrate that runtime admissibility is not limited to a single action type but can be applied consistently across heterogeneous consequential actions.

The purpose of these reference applications is not to provide production functionality but to make the execution-control boundary observable and testable in code.

## 5. Runtime Admissibility Threat Model

Agentic systems increasingly possess the ability to initiate externally consequential actions. The risks associated with increasingly autonomous systems have been discussed in the context of AI safety, trustworthy AI, and autonomous decision systems (Russell, 2019; Maybury, 2025).

Examples include:

- sending emails
- updating records
- executing workflows
- invoking APIs
- modifying infrastructure
- transferring funds

In such systems, risks emerge not only from generated content but from the execution of actions.

The threat model therefore focuses on the conditions under which consequential actions become executable.

### 5.1 Threat Assumptions

The threat model assumes that:

- agents may generate unsafe actions
- policies may be incomplete
- context may be uncertain
- users may provide adversarial instructions
- execution environments may change over time

Consequently, execution cannot be assumed admissible by default.

### 5.2 Core Threat

The primary threat is:

*A consequential action executes before admissibility has been resolved.*

Examples include:

- sensitive information sent externally
- unauthorized record modification
- unintended API invocation
- irreversible state changes
- unauthorized financial transactions

### 5.3 Threat Categories

#### Unauthorized Disclosure.

Examples include the transmission of sensitive information to unintended recipients.

Potential impact:

- confidentiality breaches
- regulatory violations
- reputational damage

#### Unauthorized Modification.

Examples include unintended updates to operational or customer records.

Potential impact:

- integrity violations
- corrupted business processes
- inconsistent system state

#### Unauthorized Execution.

Examples include workflow activation or API invocation without appropriate authorization.

Potential impact:

- operational disruption
- uncontrolled automation
- cascading failures

#### Irreversible Actions.

Examples include payments, deletions, or permanent state transitions.

Potential impact:

- financial loss
- permanent data loss
- non-recoverable consequences

### 5.4 Security Objective

The objective of runtime admissibility is not to determine whether an action is useful or desirable.

The objective is to determine whether an action is admissible.

Formally:

$$\text{Adm}(s,a,C)$$

must be resolved before:

$$\text{execute}(a)$$

The Control Before Consequence principle can therefore be expressed as:

$$\forall a : \text{execute}(a) \Rightarrow \text{Adm}(s,a,C) = \text{ALLOW}$$

This property establishes the execution-control boundary of the system.

## 6. Implications for a Trustworthy Agentic AI Roadmap

Current AI governance initiatives primarily focus on model development, model evaluation, transparency, safety testing, compliance, and accountability.

These capabilities remain essential.

However, as AI systems increasingly transition from information generation to autonomous action execution, a new governance gap emerges.

Existing governance frameworks often specify what systems should do, but provide limited mechanisms for determining whether a proposed action should be allowed to execute under runtime conditions.

This gap becomes increasingly important as agentic systems gain the ability to:

- initiate workflows
- invoke external services
- modify records
- interact with critical infrastructure
- perform financially consequential actions

The Control Before Consequence framework suggests that safe agent ecosystems require an additional governance capability:

### *Runtime Admissibility.*

Runtime admissibility introduces an execution-control layer between agent reasoning and externally consequential action.

Rather than focusing exclusively on model behavior, runtime admissibility evaluates whether a proposed action may become consequential under the current constraints.

Within a Trustworthy Agentic AI Roadmap, runtime admissibility may therefore be considered a foundational capability alongside:

- transparency
- observability
- auditability
- accountability
- safety evaluation

Unlike these capabilities, runtime admissibility directly governs execution.

As agentic systems become increasingly autonomous, execution-control mechanisms may become as important as model-level governance mechanisms.

Future roadmap activities may therefore include:

- standardized admissibility models
- interoperable policy engines
- approval workflows
- execution-control architectures
- admissibility benchmarks
- runtime governance standards

These capabilities may provide a practical path toward safer deployment of highly autonomous agentic systems.

The SAFER initiative seeks to identify critical gaps, capabilities, and governance mechanisms required for trustworthy agentic AI ecosystems. Runtime admissibility is

proposed as one such capability, specifically addressing the execution-control problem that emerges as agent autonomy increases.

## 7. Discussion and Limitations

The Control Before Consequence framework and the Runtime Admissibility Gate MVP are intended to demonstrate a new execution-control perspective for agentic AI systems.

The objective is not to provide a complete governance solution, but to identify a missing control capability that becomes increasingly important as AI systems acquire greater agency and autonomy.

A central question is whether runtime admissibility is merely another name for a policy engine or approval workflow.

This paper argues that these concepts operate at different layers.

A policy engine is a mechanism for evaluating rules.

An approval workflow is a mechanism for escalating decisions.

Runtime admissibility defines the governance decision problem itself:

May a proposed action become consequential under the current constraints?

A policy engine may be used to evaluate admissibility.

An approval workflow may be used to resolve admissibility.

However, neither is equivalent to admissibility itself.

The distinction is analogous to the relationship between authentication and authorization. Authentication establishes identity, while authorization determines whether an action is permitted.

Similarly, policy engines and approval workflows may support admissibility decisions, but they do not define the admissibility decision problem.

Runtime admissibility defines the governance decision problem, while the gate implements this decision boundary at runtime.

The contribution is not a new policy mechanism, but the explicit framing of execution control as a governance capability for agentic AI systems.

Several limitations should be acknowledged.

First, the current MVP implements only a small set of reference applications. The prototype demonstrates admissibility decisions for email actions, record updates, and API calls, but does not yet address the full diversity of consequential actions that may emerge in future agent ecosystems.

Second, the policy engine is intentionally simplified. Real-world deployments would require significantly more sophisticated policy evaluation mechanisms, contextual reasoning, and integration with organizational governance frameworks.

Third, the framework does not eliminate the need for existing governance mechanisms. Runtime admissibility

complements rather than replaces transparency, observability, auditing, accountability, security controls, and regulatory compliance.

Fourth, the current work focuses on execution control rather than agent cognition. The framework does not attempt to solve model alignment, training safety, hallucinations, bias, or broader questions of AI behavior.

Finally, significant research remains necessary to determine how admissibility policies can be standardized, validated, benchmarked, and governed across heterogeneous agent ecosystems.

These limitations should be viewed as opportunities for future research rather than weaknesses of the proposed approach.

The purpose of this work is to establish runtime admissibility as a distinct research area and governance capability for safe agentic AI systems.

## 8. Conclusion

As AI systems evolve from information-processing systems into increasingly autonomous actors, the governance challenge shifts from understanding generated content to controlling consequential action.

Current governance approaches provide important mechanisms for transparency, accountability, auditing, and compliance. However, these mechanisms often operate before action generation or after action execution. As agentic systems acquire the ability to initiate workflows, invoke external services, modify records, and interact with critical infrastructure, a new governance capability becomes necessary.

This paper introduced the Control Before Consequence framework and proposed Runtime Admissibility as a mechanism for evaluating whether a proposed action may become externally consequential under current constraints.

We formalized runtime admissibility as an execution-control problem, defined an admissibility decision space consisting of ALLOW, BLOCK, and REQUIRE\_APPROVAL, and presented a Runtime Admissibility Gate MVP that demonstrates these concepts through reference applications including email actions, record updates, and API calls.

The central contribution of this work is the argument that safe agent ecosystems require governance mechanisms that operate at the point of execution rather than exclusively at the levels of model development, content generation, or post-hoc accountability.

We therefore propose runtime admissibility as a candidate roadmap capability for trustworthy agentic AI systems.

Future work should explore standardized admissibility models, interoperable policy frameworks, approval workflows, execution-control architectures, and governance mechanisms capable of operating across heterogeneous agent ecosystems.

As agent autonomy continues to increase, determining whether an action may execute will likely become as important as determining whether a model can generate it.

## References

- [1] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete Problems in AI Safety. arXiv:1606.06565.
- [2] European Union. (2024). Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act).
- [3] IBM. (2020). Principles for Trust and Transparency in AI. IBM Corporation.
- [4] Maybury, M. T. (2025). Mitigating Biased, Brittle and Baroque Generative AI. Proceedings of the 2025 IEEE International Conference on AI and Data Analytics (ICAD). IEEE. DOI: 10.1109/ICAD65464.2025.11114038.
- [5] Maybury, M. T. (2021). Trusted Artificial Intelligence at Scale: Three Grand Challenges for AI. IEEE Computer Society.
- [6] National Institute of Standards and Technology (NIST). (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1.
- [7] U.S. Department of Defense. (2012, updated 2023). Directive 3000.09: Autonomy in Weapon Systems.
- [8] Wahlster, W. (2023). Towards Trustworthy Hybrid AI Systems. German Research Center for Artificial Intelligence (DFKI).
- [9] Rossi, F. (2019). Building Trustworthy AI. Communications of the ACM, 62(8), 40–43.
- [10] Russell, S. (2019). Human Compatible: Artificial Intelligence and the Problem of Control. Viking.
- [11] Weidinger, L., Mellor, J., Rauh, M., Griffin, C., Uesato, J., Huang, P.-S., Cheng, M., Glaese, M., Balle, B., Kasirzadeh, A., et al. (2022). Taxonomy of Risks Posed by Language Models. Proceedings of ACM FAccT 2022.
- [12] Anthropic. (2023). Constitutional AI: Harmlessness from AI Feedback. Anthropic Research.